

# C Programming - Assignment 13

---

## Q.1 Implement Descending Stack (Initialize `top = SIZE`)

### Problem Statement

Write a program to implement a **Descending Stack** using an array. Initialize the stack with `top = SIZE` and perform the following operations:

- Push
- Pop
- Peek
- Display

### Concepts Used

- Stack
- Arrays
- Functions
- Global Variables
- Conditional Statements (`if`)
- Looping (`for`)
- Menu Driven Programming

### Step-by-Step Approach

1. Define the stack array and initialize `top = SIZE`.
2. Implement the **Push** operation by decrementing `top` and inserting the element.
3. Implement the **Pop** operation by deleting the top element and incrementing `top`.
4. Implement the **Peek** operation to display the top element.
5. Implement the **Display** operation to print all stack elements.
6. Display appropriate messages for **Overflow** and **Underflow** conditions.
7. Use a menu to perform stack operations repeatedly until the user exits.

### Test Cases

#### Test Case 1

#### Operations

```
Push 10
Push 20
Display
```

### Output

```
20
10
```

---

### Test Case 2

#### Operations

```
Push 50
Peek
```

#### Output

```
Top Element = 50
```

---

### Test Case 3

#### Operations

```
Pop
Display
```

#### Output

```
Deleted = 20

Stack Elements:
10
```

---

## Q.2 Implement All Operations of Linear Queue

### Problem Statement

Write a program to implement a **Linear Queue** using an array and perform the following operations:

- Insert
- Delete
- Peek
- Display

## Concepts Used

- Queue
- Arrays
- Functions
- Front and Rear Pointers
- Conditional Statements (**if**)
- Looping (**for**)
- Menu Driven Programming

## Step-by-Step Approach

1. Initialize **front** and **rear** to **-1**.
2. Implement **Insert** operation.
3. Implement **Delete** operation.
4. Implement **Peek** operation.
5. Implement **Display** operation.
6. Handle **Queue Overflow** and **Queue Underflow** conditions.
7. Continue operations until the user selects Exit.

## Test Cases

### Test Case 1

#### Operations

```
Insert 10
Insert 20
Insert 30
Display
```

#### Output

```
10 20 30
```

### Test Case 2

#### Operations

```
Delete
Display
```

#### Output

Deleted = 10

20 30

---

### Test Case 3

#### Operations

Peek

#### Output

Front Element = 20

---

## Q.3 Implement All Operations of Circular Queue

### Problem Statement

Write a program to implement a **Circular Queue** using an array and perform the following operations:

- Insert
- Delete
- Peek
- Display

### Concepts Used

- Circular Queue
- Arrays
- Functions
- Front and Rear Pointers
- Modulus Operator (%)
- Conditional Statements (if)
- Menu Driven Programming

### Step-by-Step Approach

1. Initialize **front** and **rear** to **-1**.
2. Implement **Insert** using circular indexing.
3. Implement **Delete** operation.
4. Implement **Peek** operation.
5. Implement **Display** operation.

6. Detect **Queue Overflow** using circular condition.
7. Detect **Queue Underflow** when the queue becomes empty.
8. Continue operations until Exit is selected.

## Test Cases

### Test Case 1

#### Operations

```
Insert 5  
Insert 10  
Insert 15  
Display
```

#### Output

```
5 10 15
```

---

### Test Case 2

#### Operations

```
Delete  
Display
```

#### Output

```
Deleted = 5  
  
10 15
```

---

### Test Case 3

#### Operations

```
Peek
```

#### Output

Front Element = 10

---

SUNBEAM